Projektowanie aplikacji webowych opartych o mikrousługi

LAB2 — Zapisywanie danych w bazie i obsługa cache

Projektowanie aplikacji webowych opartych o mikrousługi

Czas: 90–120 minut zajęć; przygotowanie środowiska przed zajęciami, dokończenie implementacji, testów i README po zajęciach (około 60–120 minut)

Technologia: Python + FastAPI + SQLAlchemy + SQLite + Redis

Forma: indywidualnie lub w parach, w repozytorium projektu z LAB1

Rezultat: dotychczasowa usługa z trwałymi danymi, cache wybranego odczytu i testami

1. Cele i zakres laboratorium

Po laboratorium powinieneś:

- wyjaśnić różnicę między pamięcią procesu, bazą danych i cache,
- przenieść dane swojej usługi do bazy bez przypadkowej zmiany API,
- zachować regułę biznesową z LAB1 i brak zmiany stanu po odrzuconej operacji,
- sprawdzić trwałość zapisu po restarcie,
- zastosować cache-aside, TTL i unieważnianie,
- przetestować odczyt i zapis przy niedostępnym Redis,
- wyjaśnić konsekwencje nieaktualnych danych.

Zachowanie ciągłości projektu z LAB1

LAB1	Rozwinięcie na LAB2
Własny temat, aktor i pojęcia domeny	Zachowujesz granicę pierwszej usługi
Lista, szczegóły, zapis i health	Zachowujesz endpointy, identyfikatory i JSON
Dane w pamięci	Trwały zapis do bazy
Reguła biznesowa, 404, 409, 422	Te same zachowania po zmianie magazynu
Świeża aplikacja w testach	Osobna baza i cache dla testów
README i plan drugiej usługi	Dopisujesz decyzje o danych i cache

1.1. Podstawowe pojęcia persystencji i cache

Persystencja oznacza zachowanie danych poza czasem życia procesu. SQLite zapisuje bazę w pliku. Restart Uvicorna nie usuwa tego pliku.

SQLAlchemy jest biblioteką dostępu do bazy. ORM (*Object-Relational Mapping*) odwzorowuje obiekty Pythona na wiersze tabel. Model Pydantic opisuje kontrakt API; model ORM opisuje zapis danych.

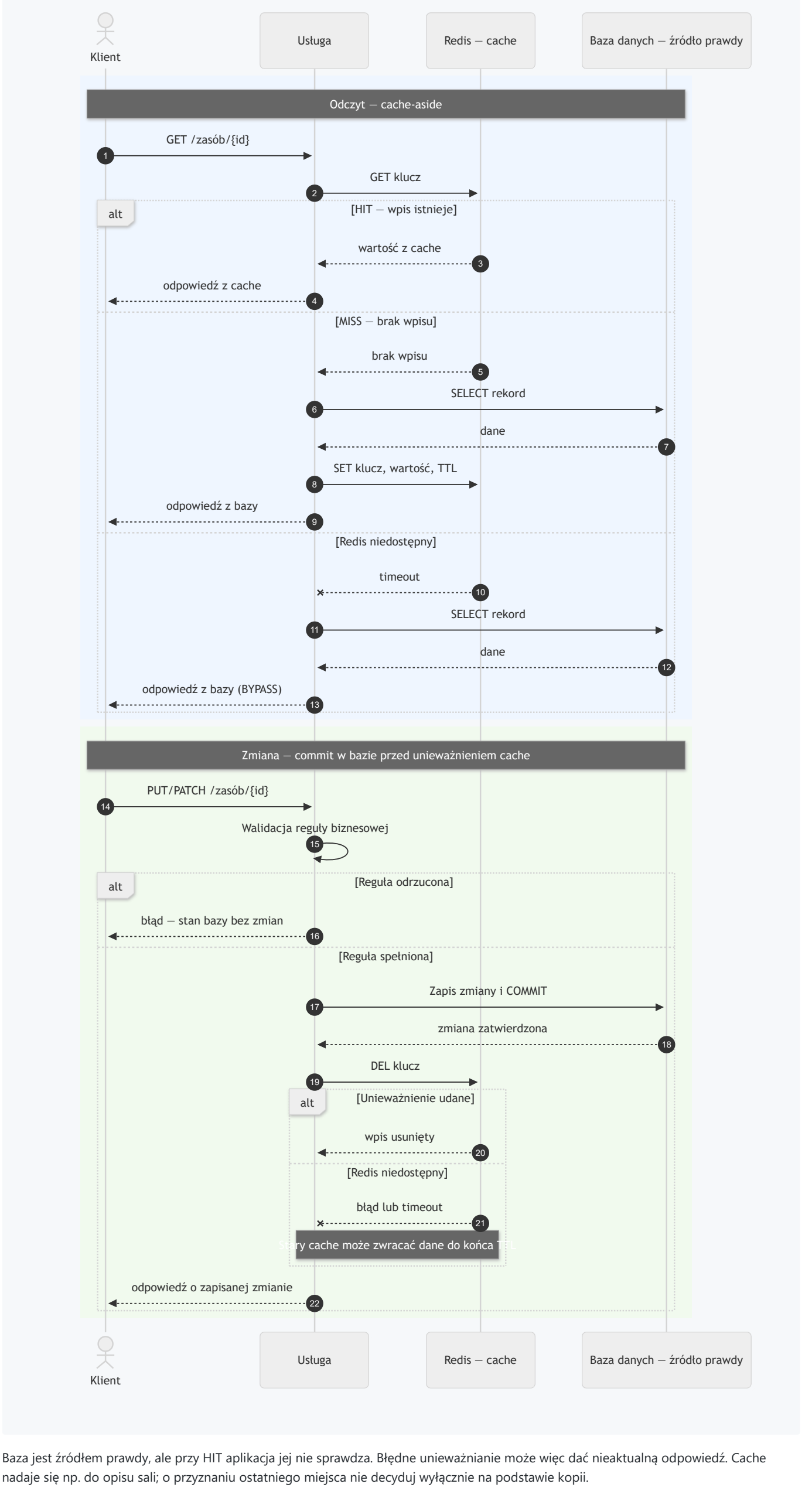
Sesja obsługuje pracę z bazą w ramach operacji. **Transakcja** grupuje zmiany: commit je zatwierdza, rollback wycofuje. Sesji nie współdzielimy między równoległymi żądaniami. **Silnik** (`Engine`) zarządza połączeniami z bazą; nie jest bazą ani sesją. Może być wspólny dla instancji aplikacji.

Cache to odtwarzalna kopia wybranych danych. Redis działa jako osobny proces, więc może przeżyć restart aplikacji. Nie zakładamy jednak obecności wpisu: może wygasnąć, zostać usunięty lub utracony.

TTL (*Time To Live*) to czas życia wpisu. **HIT** oznacza znalezienie go, **MISS** — brak. W naszym przykładzie **BYPASS** oznacza pominięcie niedostępnego cache.

Cache-aside to wzorzec, w którym aplikacja najpierw sprawdza cache. Przy **HIT** zwraca znalezioną kopię. Przy **MISS** pobiera dane z bazy, zapisuje je w cache z określonym TTL i zwraca wynik. Po zatwierdzeniu zmiany w bazie usuwa odpowiadający wpis z cache, aby kolejny odczyt pobrał aktualne dane. Baza pozostaje źródłem prawdy.

Przepływ cache-aside — odczyt i zmiana danych:



Baza jest źródłem prawdy, ale przy HIT aplikacja jej nie sprawdza. Błędne unieważnianie może więc dać nieaktualną odpowiedź. Cache nadaje się np. do opisu sali; o przyznaniu ostatniego miejsca nie decyduj wyłącznie na podstawie kopii.

Analiza odpowiedzialności za dane

Dotyczą znanego z LAB1 systemu paczkomatów:

1. Co dzieje się ze zmianą statusu skrytki po restarcie wersji z LAB1?
2. Która usługa powinna zapisywać stan skrytki?
3. Czy historia paczki należy do tych samych danych?
4. Czy nieaktualny opis paczkomatu ma te same konsekwencje co nieaktualna informacja o wolnej skrytce?

Decyzje o własnym cache podejmiesz w części B, po analizie przykładu.

---

2. Organizacja realizacji laboratorium

LAB2 obejmuje krótki przykład wspólny oraz pracę nad usługą rozwijaną w LAB1. Przykład służy poznaniu mechanizmu; pełne odtworzenie przykładowej usługi `Locker Service` nie jest wymagane.

Praca jest kontynuowana w tym samym repozytorium. Należy zachować temat, kontrakt API, reguły biznesowe i fabrykę `create_app()` z LAB1. Przed przebudową magazynu danych należy utworzyć commit działającej wersji. Port z LAB1 można zachować; w razie konfliktu należy wybrać wolny port i odpowiednio zmienić polecenia.

LAB2 dotyczy pierwszej usługi, więc nie wymaga uruchamiania drugiej. Wymóg co najmniej dwóch różnych mikrousług na osobę dotyczy całego projektu semestralnego.

Proponowany harmonogram (120 minut): 10 minut teorii, 20 minut przykładu, 15 minut decyzji projektowych, 40 minut pracy nad bazą, 25 minut pracy nad cache i testami oraz 10 minut podsumowania. Redis należy przygotować przed zajęciami. Przy bloku 90-minutowym część implementacji trzeba dokończyć po zajęciach; termin oddania określa prowadzący.

Punkt kontrolny na koniec zajęć: trwały zapis we własnej usłudze oraz pierwszy scenariusz MISS → HIT. Testy awarii i dokumentację można dokończyć po zajęciach, ale pozostają wymaganiami oddawanego etapu.

W pracy w parach należy zmieniać role osoby implementującej i sprawdzającej. Każda osoba powinna umieć wyjaśnić rozwiązanie i samodzielnie wprowadzić drobną zmianę podczas odbioru.

Weryfikacja kodu wspomaganego przez AI

AI jest dozwolone na zasadach LAB1. Dopisz do README zakres pomocy i sposób weryfikacji albo „nie korzystano". Wygenerowany kod musi przejść testy; decyzje o modelu, transakcjach i cache musisz umieć wyjaśnić.

---

3. Przygotowanie środowiska i zależności

Polecenia w instrukcji są przeznaczone dla PowerShella na systemie Windows.

Instalacja i konfiguracja wymaganych narzędzi

Przed rozpoczęciem upewnij się, że masz przygotowane:

1. **Python 3.12 lub nowszy:**
   - Sprawdź wersję: `python --version` .
   - W razie potrzeby zainstaluj przez winget: `winget install Python.Python.3.12` (po instalacji uruchom nowy terminal) lub pobierz instalator z python.org (pamiętając o zaznaczeniu opcji *Add python.exe to PATH*).
2. **Serwer Redis — wybierz jeden wariant:**
   - **Opcja A (rekomendowana) — Docker Desktop:** pobierz z docker.com lub zainstaluj przez `winget install Docker.DockerDesktop` . Dla wariantu z backendem WSL2 potrzebujesz włączonej wirtualizacji sprzętowej i skonfigurowanego WSL2.
   - **Opcja B — WSL2 (bez Dockera):** w terminalu Ubuntu/Debian w WSL zainstaluj i uruchom: `sudo apt update && sudo apt install redis-server && sudo service redis-server start` .
   - **Opcja C — serwis lokalny Windows:** np. Memurai (`winget install Memurai.Memurai`).
   - *Uwaga:* testy automatyczne w projekcie korzystają z pamięciowej atrapy `MemoryRedis` , więc nie wymagają działającego Dockera ani serwera Redis.
3. **Przeglądarka bazy SQLite (opcjonalnie, zalecane):**
   - Pozwala wygodnie podglądać tabele i rekordy w plikach `.db` :
     - Rozszerzenie do VS Code: **SQLite Viewer** (Florian Klampfer) — bezpośredni podgląd tabel w edytorze.
     - Samodzielny program: **DB Browser for SQLite** (`winget install SQLiteBrowser.SQLiteBrowser`).
     - **DataGrip** (JetBrains) — samodzielny klient bazodanowy do przeglądania plików SQLite i wykonywania zapytań.

Pobranie i struktura kodu pomocniczego

Pobierz kod pomocniczy LAB2 — ZIP i rozpakuj **obok własnego projektu**:

Plik	Do czego służy
`database.py`	Silnik i fabryka sesji SQLAlchemy
`cache.py`	JSON w Redis, TTL, timeouty i obsługa awarii
`demo.py`	Uruchamialny przykład zapisu, odczytu i unieważnienia
`tests/test_storage.py`	Testy trwałości, rollbacku i cache
`README.md`	Uruchomienie oraz wskazówki integracji
`README-LAB2-template.md`	Sekcje do dopisania do swojego README

Paczka pokazuje mechanizm na danych paczkomatu. Nie jest pełnym zamiennikiem `Locker Service` z LAB1: nie udostępnia HTTP ani operacji skrytek. Własne API i reguły rozwijasz w swoim repozytorium.

Uruchamianie i weryfikacja serwera cache (Redis)

W dalszej części instrukcji wykonasz operacje sprawdzenia połączenia (`PING`), usunięcia klucza (`DEL`), sprawdzenia czasu życia (`TTL`) oraz symulacji awarii (zatrzymania i wznowienia serwera). Użyj poleceń właściwych dla wybranego środowiska:

Operacja	Opcja A: Docker Desktop	Opcja B: WSL2	Opcja C: Memurai / Windows
Uruchomienie serwera	`docker run --name lab2-redis -p 127.0.0.1:6379:6379 -d redis:7-alpine`	`wsl sudo service redis-server start`	Działa automatycznie jako usługa Windows
Test połączenia (PING)	`docker exec lab2-redis redis-cli ping`	`wsl redis-cli ping`	`memurai-cli ping`
Usunięcie klucza (DEL)	`docker exec lab2-redis redis-cli DEL <klucz>`	`wsl redis-cli DEL <klucz>`	`memurai-cli DEL <klucz>`
Czas życia klucza (TTL)	`docker exec lab2-redis redis-cli TTL <klucz>`	`wsl redis-cli TTL <klucz>`	`memurai-cli TTL <klucz>`
Zatrzymanie (awaria)	`docker stop lab2-redis`	`wsl sudo service redis-server stop`	`Stop-Service Memurai` (PowerShell jako Admin)
Wznowienie po awarii	`docker start lab2-redis`	`wsl sudo service redis-server start`	`Start-Service Memurai` (PowerShell jako Admin)

Po komendzie ping oczekuj odpowiedzi `PONG` . Jeśli kontener już istnieje, użyj `docker start lab2-redis` . W poleceniach `DEL` i `TTL` zastąp `<klucz>` pełną nazwą swojego wpisu; w przykładzie paczkomatu użyj zmiennej `$demoKey` zdefiniowanej w kroku 1. `DEL` zwraca `1` , jeśli usunął wpis, i `0` , jeśli klucz nie istniał.

Instalacja zależności i uruchomienie testów przykładu

W terminalu przejdź do rozpakowanego katalogu z `requirements.txt` :

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$env:DATABASE_URL = 'sqlite:///./demo.db'
$env:CACHE_PREFIX = 'lab2:album12345:demo' # zastąp 12345 swoim numerem albumu
$env:CACHE_TTL_SECONDS = '60'
.\.venv\Scripts\python.exe -m pytest -q
```

Nie trzeba aktywować środowiska. Zmienne obowiązują w bieżącym terminalu. Testy automatyczne używają kontrolowanego zamiennika Redis; rzeczywiste połączenie sprawdzisz poniżej.

Przed uruchomieniem demo ustaw adres lokalnego Redis: `$env:REDIS_URL = 'redis://localhost:6379/0'` . Nie nadpisuj go później innym adresem. Sekretów nie zapisuj w repozytorium. Przy zmianie z demo na własną usługę nadaj jej odrębny prefiks, np. `lab2:album12345:room-service` .

`demo.py` czyta zmienne przy każdym uruchomieniu. Działający Uvicorn nie przejmuje zmian zmiennych z terminala: zatrzymaj go przez Ctrl+C i uruchom ponownie **w terminalu z nową konfiguracją**. Samo `--reload` tego nie zapewnia.

---

4. Część A — analiza persystencji i cache na przykładzie

Zachowanie kontraktu i identyfikatorów z LAB1

Paczkomat nadal ma tekstowy identyfikator `KRK-1024` oraz pola `name` , `address` , `city` , `status` . Nie zmieniamy typu ID ani JSON ze względu na dodanie bazy.

Fragment `demo.py` :

```python
class LockerRow(Base):
    __tablename__ = "lockers"
    id: Mapped[str] = mapped_column(String(32), primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    address: Mapped[str] = mapped_column(String(200))
    city: Mapped[str] = mapped_column(String(80))
    status: Mapped[str] = mapped_column(String(20), default="online")
```

Pełne importy i konfiguracja są w paczce. Klucz główny zapewnia unikalność ID. Zwykły indeks sam nie zapewnia unikalności. SQLite nie egzekwuje długości `String(n)` jak walidator — zachowaj ograniczenia Pydantic.

Krok 1 — weryfikacja zapisu i odczytu w bazie

```
.\.venv\Scripts\python.exe demo.py seed
.\.venv\Scripts\python.exe demo.py get
.\.venv\Scripts\python.exe demo.py get
```

Przy pustym, dostępnym cache zobaczysz MISS i HIT. Każde polecenie uruchamia nowy proces, a rekord pozostaje w `demo.db` . `seed` nie nadpisuje istniejącego paczkomatu.

Jeśli pierwszy odczyt daje HIT, pozostał wcześniejszy wpis. Usuń wyłącznie klucz przykładu:

```
$demoKey = "${env:CACHE_PREFIX}:locker:KRK-1024"
# Wykonaj tylko polecenie swojego wariantu:
docker exec lab2-redis redis-cli DEL $demoKey
# WSL2: wsl redis-cli DEL $demoKey
# Memurai: memurai-cli DEL $demoKey
```

Krok 2 — aktualizacja danych i unieważnienie cache

```
.\.venv\Scripts\python.exe demo.py rename --name 'Nowa Nazwa'
.\.venv\Scripts\python.exe demo.py get
.\.venv\Scripts\python.exe demo.py get
```

Oczekuj `cache_invalidated=True` , potem MISS z nową nazwą i HIT z tą samą nazwą.

W `LockerRepository.rename` znajdziesz:

```python
with self.sessions.begin() as db:
    row = db.get(LockerRow, locker_id)
    if row is None:
        raise LookupError(locker_id)
    row.name = name
    result = payload(row)
return result
```

Poprawne wyjście z bloku zatwierdza zmianę, a wyjątek wycofuje transakcję. `CachedLockers.rename` usuwa wpis dopiero po powrocie z tej metody.

Zmiana nazwy jest ćwiczeniem technicznym. Nie zastępuje reguły przejść statusów skrytek z LAB1.

Krok 3 — weryfikacja TTL i obsługi niedostępności cache

```
# Sprawdzenie TTL (Docker):
$demoKey = "${env:CACHE_PREFIX}:locker:KRK-1024"
docker exec lab2-redis redis-cli TTL $demoKey
# Dla innych środowisk użyj odpowiednika z tabeli (np. wsl redis-cli TTL ... lub memurai-cli TTL ...)
```

TTL podaje pozostałe sekundy; `-2` oznacza brak klucza, `-1` brak terminu wygaśnięcia. Po wygaśnięciu kolejny odczyt daje MISS.

Zatrzymaj **własną lokalną instancję Redis**:

```
# Docker:
docker stop lab2-redis
.\.venv\Scripts\python.exe demo.py get
.\.venv\Scripts\python.exe demo.py rename --name 'Zapis bez cache'
docker start lab2-redis

# WSL2 odpowiednik: wsl sudo service redis-server stop
# Po teście: wsl sudo service redis-server start
# Memurai odpowiednik: Stop-Service Memurai
# Po teście: Start-Service Memurai
```

Odczyt daje BYPASS i dane z bazy. Zapis zostaje zatwierdzony mimo nieudanego unieważnienia. Zatrzymanie serwera nie gwarantuje zachowania starych wpisów po restarcie.

Aby odtworzyć **stary wpis po nieudanym unieważnieniu**, pozostaw Redis uruchomiony i odłącz tylko klienta. Wykonaj całość w jednym terminalu, w czasie krótszym niż 300 sekund:

```
$workingRedisUrl = $env:REDIS_URL
$previousTtl = $env:CACHE_TTL_SECONDS
$env:CACHE_TTL_SECONDS = '300'
try {
    .\.venv\Scripts\python.exe demo.py rename --name 'Przed awaria'
    .\.venv\Scripts\python.exe demo.py get # MISS, wypełnia cache
    $env:REDIS_URL = 'redis://127.0.0.1:9999/0' # port musi być nieużywany
    .\.venv\Scripts\python.exe demo.py get # BYPASS
    .\.venv\Scripts\python.exe demo.py rename --name 'Po awarii'
    $env:REDIS_URL = $workingRedisUrl
    .\.venv\Scripts\python.exe demo.py get # HIT: stara nazwa 'Przed awaria'
} finally {
    $env:REDIS_URL = $workingRedisUrl
    $env:CACHE_TTL_SECONDS = $previousTtl
}
```

Przed odłączeniem potwierdź udane unieważnienie oraz MISS; jeśli widzisz BYPASS, najpierw napraw połączenie. Po wygaśnięciu wpisu (sprawdź TTL własnego klucza) ponów GET: oczekuj MISS i nazwy `Po awarii` . Przywrócenie konfiguracji TTL nie skraca czasu życia już zapisanego wpisu. We własnym API analogiczny eksperyment wymaga restartu Uvicorna po każdej zmianie adresu.

Ograniczenie: po przywróceniu łączności stary wpis może istnieć aż do wygaśnięcia TTL. W paczce jest test tego przypadku. Commit w bazie i usunięcie klucza Redis nie są jedną transakcją. Prosty cache-aside nie zapewnia też ścisłej spójności przy równoległych zapisach i odczytach.

Analiza skutków niedostępności cache (Redis)

1. Dlaczego dane przetrwały uruchomienie nowego procesu?
2. Co pokazuje HIT, a czego nie dowodzi o wydajności?
3. Dlaczego błąd zapisu nie powinien zatwierdzić części danych?
4. Dlaczego obsługujemy awarię GET, SET i DELETE w Redis?
5. Co grozi po nieudanym unieważnieniu?

---

5. Część B — rozbudowa własnej usługi z LAB1

To główna część laboratorium. Zastosuj mechanizm z przykładu do dotychczasowej domeny.

Krok 1 — weryfikacja stanu początkowego usługi

Uruchom dotychczasowe testy i usługę:

```
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8101
```

W drugim terminalu sprawdź health i własne endpointy. Zmień dane przez API, potwierdź GET, zatrzymaj proces, uruchom go ponownie i powtórz odczyt. Zapisz wynik LAB1. Jeśli już masz bazę, opisz istniejącą persystencję zamiast celowo ją usuwać.

Zachowaj kontrakt: metody, ścieżki, ID, JSON i błędy. Uzupełniasz README z LAB1.

Krok 2 — zaprojektowanie modelu danych

Zachowaj aktora, słownik domeny i regułę biznesową. Uzupełnij:

Decyzja	Własna odpowiedź i uzasadnienie
Jakie dane muszą przetrwać restart?	...
Jakie tabele i powiązania są potrzebne?	...
Która usługa może je zapisywać?	...
Których danych drugiej usługi tu nie zapisujemy?	...
Co chroni baza, a co reguła biznesowa?	...
Jaki odczyt cache'ujemy i jak długo może być nieaktualny?	...
Jaki prefiks, klucz, TTL i sposób unieważnienia wybieramy?	...

Wszystkie dotychczasowe operacje dotyczące przenoszonych danych muszą korzystać z tego samego trwałego magazynu. Baza obok nadal używanego słownika nie spełnia tego wymagania.

Krok 3 — podłączenie bazy z zachowaniem kontraktu API

Dopisz do istniejących zależności `SQLAlchemy>=2.0,<2.1` i `redis>=5,<7` . Zachowaj zależności LAB1 i zainstaluj zaktualizowany plik.

Możesz wykorzystać `database.py` z paczki. Dodaj modele ORM w osobnym module, np. `db_models.py` , aby nie nadpisać modeli Pydantic.

```
my-service/
├── app/
│   ├── main.py          # dotychczasowe create_app i API
│   ├── models.py        # modele Pydantic z LAB1
│   ├── db_models.py     # nowe tabele
│   ├── database.py
│   ├── repository.py
│   └── cache.py
├── tests/
├── requirements.txt
└── README.md
```

Użyj np. `sqlite:///./app.db` . Ścieżka jest względna do katalogu uruchomienia. Dodaj `*.db` i pliki pomocnicze SQLite do `.gitignore` . Zachowaj w repozytorium kod tworzący schemat. Dostarczony w paczce `database.py` włącza obsługę kluczy obcych (`PRAGMA foreign_keys=ON;`), która w SQLite jest domyślnie wyłączona — dzięki temu ograniczenia relacji (jeśli Twoja usługa ma powiązane tabele) są rygorystycznie sprawdzane.

W `create_app()` przygotuj silnik i fabrykę sesji. Przekazuj konfigurację jako argument, aby test mógł wskazać inną bazę. `lifespan` uruchamia przygotowanie zasobów przed obsługą żądań i sprzątanie po zakończeniu pracy aplikacji.

Poniższy szkielet w `app/main.py` pokazuje wyłącznie połączenie elementów. `RoomRepository` jest **Twoją klasą do napisania** (konstruktor przyjmuje fabrykę sesji); dostosuj nazwę do domeny. Zachowaj endpointy i modele z LAB1:

```python
import os
from contextlib import asynccontextmanager

from fastapi import FastAPI
from app.database import Base, make_database
from app.cache import JsonCache, make_redis
from app import db_models  # rejestruje tabele we wspólnym Base.metadata
from app.repository import RoomRepository


def create_app(database_url=None, redis_client=None):
    @asynccontextmanager
    async def lifespan(app):
        engine, sessions = make_database(
            database_url or os.getenv("DATABASE_URL", "sqlite:///./app.db")
        )
        client = None
        try:
            client = redis_client if redis_client is not None else make_redis(
                os.getenv("REDIS_URL", "redis://localhost:6379/0")
            )
            Base.metadata.create_all(engine)
            app.state.engine = engine
            app.state.redis = client
            app.state.repository = RoomRepository(sessions)
            app.state.cache = JsonCache(
                client,
                prefix=os.environ["CACHE_PREFIX"],
                ttl=int(os.getenv("CACHE_TTL_SECONDS", "60")),
                resource_name="room",
            )
            yield
        finally:
            try:
                if client is not None:
                    client.close()
            finally:
                engine.dispose()

    app = FastAPI(lifespan=lifespan)
    # Tutaj zachowaj/rejestruj endpointy z LAB1.
    return app
```

Ustaw `CACHE_PREFIX` przed startem; w testach ustawisz go przez `monkeypatch` . Ten wariant przekazuje aplikacji odpowiedzialność za zamknięcie klienta Redis, także wstrzykniętego. Każdej instancji testowej przekaż nowy `MemoryRedis` z paczki (ma metodę `close()`). Repozytorium powinno zamykać sesję po każdej operacji; samo `engine.dispose()` nie zastępuje zamknięcia aktywnych sesji. Zachowaj dotychczasowy sposób uruchamiania fabryki z LAB1.

`create_all` tworzy brakujące tabele, ale nie aktualizuje istniejących kolumn. Nie kasuj bazy przy każdym uruchomieniu. Migracje Alembic są rozszerzeniem.

Krok 4 — przeniesienie operacji i reguły biznesowej do bazy

Zaimplementuj listę, szczegóły i dotychczasowy zapis. Jeśli LAB1 miał tylko POST, dodaj jedną uzasadnioną modyfikację albo usunięcie do testowania unieważnienia. Przy samym PATCH wystarczą dane początkowe, które nie nadpisują istniejącego stanu.

Kolejność: odczyt stanu → sprawdzenie reguły → zmiana → commit. Repozytorium z paczki pokazuje sesję na operację.

Zachowaj 404, 409 i 422. Mapuj znany konflikt unikalności na 409; nie zamieniaj każdej awarii bazy w konflikt. Po błędzie wycofaj transakcję. Ponowny odczyt ma pokazać poprzedni stan.

Zachowaj istniejący `response_model` . Do cache wkładaj serializowaną odpowiedź API, np. `response.model_dump(mode="json")` , a nie obiekt sesji lub surowy obiekt ORM.

Dowód: zapis → GET → zatrzymanie Uvicorna → uruchomienie → GET. Zmiana ma przetrwać. Wykonaj odczyt bez starego cache, aby dowieść zapisu do bazy.

Krok 5 — zastosowanie cache-aside dla wybranego odczytu

Wykorzystaj `JsonCache` albo napisz własny adapter. Dostosuj klucze do domeny: `JsonCache(client, prefix="lab2:12345:room-service", resource_name="room")` tworzy np. `lab2:12345:room-service:room:R-12` . Domyślny `resource_name="locker"` dotyczy wyłącznie przykładu paczkomatów.

Poniższy fragment umieść wewnątrz `create_app()` , przed `return app` (dopasuj wcięcie). `RoomResponse` i `RoomUpdate` zaimportuj ze swoich modeli. Obiekty repozytorium i cache pobieramy z `request.app.state` , ponieważ powstają dopiero podczas startu aplikacji:

```python
from fastapi import HTTPException, Request, Response


@app.get("/rooms/{room_id}", response_model=RoomResponse)
def get_room(room_id: str, request: Request, response: Response):
    repository = request.app.state.repository
    cache = request.app.state.cache
    cached, source = cache.get(room_id)
    if cached is not None:
        response.headers["X-Cache"] = source
        return cached

    try:
        result = repository.get(room_id)
    except LookupError as exc:
        raise HTTPException(status_code=404, detail="Room not found") from exc
    if result is None:
        raise HTTPException(status_code=404, detail="Room not found")
    payload = RoomResponse.model_validate(result).model_dump(mode="json")
    if source != "BYPASS" and not cache.put(room_id, payload):
        source = "BYPASS"
    response.headers["X-Cache"] = source
    return payload
```

Repozytorium może zwracać słownik (jak w paczce) lub model `RoomResponse` . Konwersja przez `RoomResponse.model_validate` zapewnia wspólny kontrakt przed serializacją. Przy zwracaniu obiektu ORM skonfiguruj w modelu Pydantic `from_attributes=True` . Brak zasobu jest obsłużony zarówno dla `LookupError` z przykładu, jak i dla wyniku `None` ; inne błędy nie są zamieniane na 404. Przy synchronicznym SQLAlchemy i kliencie Redis używaj endpointów `def` , jak w LAB1.

Po udanym zapisie unieważniaj właściwy klucz. Kolejność operacji: **najpierw commit w bazie, a dopiero potem delete w cache**:

```python
@app.patch("/rooms/{room_id}", response_model=RoomResponse)
def update_room(room_id: str, payload: RoomUpdate, request: Request):
    repository = request.app.state.repository
    cache = request.app.state.cache
    # 1. Sprawdzenie reguły i zapis w bazie z zatwierdzeniem transakcji:
    try:
        updated = repository.update(room_id, payload)
    except LookupError as exc:
        raise HTTPException(status_code=404, detail="Room not found") from exc
    except ValueError as exc:  # naruszenie reguły biznesowej z LAB1 (mapowane na 409)
        raise HTTPException(status_code=409, detail=str(exc)) from exc

    if updated is None:
        raise HTTPException(status_code=404, detail="Room not found")

    # 2. Unieważnienie klucza PO zatwierdzeniu zmiany w bazie:
    cache.delete(room_id)
    return RoomResponse.model_validate(updated)
```

Dlaczego ta kolejność? Jeśli transakcja w bazie zostanie wycofana przez błąd lub regułę biznesową, cache nie powinien zostać niepotrzebnie usunięty. Z kolei ewentualna awaria Redis podczas `cache.delete()` nie cofa zatwierdzonej operacji w bazie danych. Cache listy wymaga także uwzględnienia zmian elementów i filtrów — dla minimum projektowego wystarczy cache pojedynczego zasobu.

Dowód: `curl.exe -i http://localhost:8101/rooms/R-12` pokazuje status, JSON i `X-Cache` . Dopasuj adres. Możesz użyć logów, jeśli jednoznacznie odróżnisz HIT, MISS i BYPASS.

Redis nie musi być szybszy od lokalnego SQLite dla jednego rekordu. Wykaż pominięcie odczytu repozytorium przy HIT; pomiar czasu jest rozszerzeniem.

Krok 6 — weryfikacja zachowania podczas awarii cache

Obsłuż brak Redis podczas odczytu, zapisywania kopii i unieważniania. Ustaw krótkie timeouty i ogranicz ponowienia, jak w paczce. Błąd cache nie może cofnąć już zatwierdzonej operacji bazy.

Pokaż normalny scenariusz zmiany i scenariusz nieudanego unieważnienia. Opisz możliwość starej wartości po powrocie Redis. Jeśli domena nie toleruje opóźnienia, wybierz do cache inny rodzaj danych.

Zadanie rozszerzające — implementacja endpointu gotowości `/ready`

Zachowaj `/health` i jego kontrakt z LAB1. Opcjonalnie dodaj osobny `/ready` : sprawna baza oznacza gotowość, a niedostępny opcjonalny cache — stan `degraded` , nadal z HTTP 200. Awaria bazy daje 503. Fragment korzysta z zasobów powyższego szkieletu; umieść endpoint wewnątrz fabryki. To rozszerzenie, nie dodatkowy warunek zaliczenia.

```python
from fastapi import Request
from fastapi.responses import JSONResponse
from sqlalchemy import text
from sqlalchemy.exc import SQLAlchemyError
from redis.exceptions import RedisError


@app.get("/ready")
def readiness(request: Request):
    try:
        with request.app.state.engine.connect() as connection:
            connection.execute(text("SELECT 1"))
        db_ok = True
    except SQLAlchemyError:
        db_ok = False
    try:
        cache_ok = bool(request.app.state.redis.ping())
    except RedisError:
        cache_ok = False

    status = "ok" if (db_ok and cache_ok) else ("degraded" if db_ok else "error")
    status_code = 200 if db_ok else 503

    return JSONResponse(
        status_code=status_code,
        content={
            "service": "room-service",  # wpisz nazwę swojej usługi
            "status": status,
            "database": "ok" if db_ok else "down",
            "cache": "ok" if cache_ok else "down",
        },
    )
```

Implementacja rozwiązania na podstawie dokumentacji (EK_03)

Wykorzystaj minimum dwa fragmenty oficjalnej dokumentacji:

- SQLAlchemy: sesje i transakcje,
- redis-py: połączenie z serwerem,
- FastAPI: podmienianie zależności w testach.

W README zapisz problem, odnośnik, zastosowanie i sposób sprawdzenia. Sam wykaz linków nie wystarcza.

> **Wskazówka:** Szczegółowy przewodnik dotyczący pracy z agentami AI, plików `SKILL.md` oraz przykładowych promptów znajdziesz w rozdziale o dokumentacji i asystentach AI.

Krok 7 — implementacja i uruchomienie testów

Dostosuj testy LAB1: scenariusze API pozostają, ale fixture tworzy oddzielną bazę w `tmp_path` . Nowe `create_app()` nie oznacza już automatycznie pustego stanu — decyduje adres bazy.

Dodaj minimum **cztery własne testy automatyczne**:

Test	Co musi wykazać
Trwałość	Zapis, zamknięcie instancji, nowa instancja z tą samą bazą i pustym cache, poprawny odczyt
HIT/MISS	Pierwszy odczyt z bazy, drugi z cache; przy HIT repozytorium nie jest wywołane
Unieważnienie	Zapełnienie cache, zmiana przez API, następny GET zwraca nową wartość (albo 404 po DELETE)
Awaria cache	Niedostępny Redis nie blokuje odczytu ani zapisu do sprawnej bazy

Zachowaj testy walidacji, konfliktu i reguły z LAB1. Testy z ZIP nie zastępują testów Twojego API.

Wskazówki implementacji czterech testów w `TestClient(app)` :

- **Trwałość:** zapisz rekord przez POST w pierwszym bloku `with TestClient(app1):` . Przy API z samym PATCH zmień istniejący rekord na wartość inną niż początkowa. Po wyjściu z bloku uruchom `app2` z tym samym plikiem bazy w `tmp_path` , ale **nowym, pustym** `MemoryRedis` . GET ma zwrócić zapisany stan z bazy, nie z kopii. Inicjalizacja nie może odtwarzać zmienionej wartości przy każdym starcie.
- **HIT/MISS:** pierwszy odczyt przez GET sprawdza nagłówek `response.headers.get("X-Cache") == "MISS"` , kolejny ten sam odczyt potwierdza `response.headers.get("X-Cache") == "HIT"` . Aby dowieść, że przy HIT baza nie została powtórnie odpytana (zgodnie z tabelą wymagań), użyj licznika wywołań przez `monkeypatch` (jak w `test_miss_hit_and_invalidation` z paczki). Wymaga to udostępnienia instancji repozytorium w aplikacji (np. `app.state.repository` lub parametru fabryki `create_app`), po czym zliczasz wywołania metody repozytorium i upewniasz się, że wykonała tylko 1 odczyt po obu zapytaniach HTTP.
- **Unieważnienie:** po odczycie (HIT) wykonaj PATCH/PUT; następny GET ma zwrócić nową wartość i MISS. Jeśli wybraną operacją jest DELETE, sprawdź usunięcie klucza i odpowiedź 404 przy następnym GET. Nie wymagamy nagłówka MISS przy 404 ani przechowywania odpowiedzi 404 w cache.
- **Awaria cache:** podmień klienta Redis na zamiennik zgłaszający awarię (`client.down = True` , jak w `MemoryRedis` z paczki pomocniczej) i upewnij się, że GET nadal zwraca kod 200 z nagłówkiem `"BYPASS"` , a zapis zostaje zatwierdzony w bazie danych.

Jeśli wybrałeś logi zamiast `X-Cache` , w tych samych testach sprawdzaj znaczniki MISS/HIT/BYPASS przez `caplog` zamiast nagłówków. Nadal wymagane są asercje odpowiedzi API, stanu bazy i liczby odczytów repozytorium. Sam adapter z paczki nie loguje wszystkich tych znaczników — dodaj je w swojej warstwie obsługi odczytu.

Przykład fixture łączącej bazę w `tmp_path` oraz zamiennik Redis bez konieczności uruchamiania Dockera w testach automatycznych:

```python
import pytest
from fastapi.testclient import TestClient
from app.main import create_app
# W tym przykładzie klasę MemoryRedis wklej powyżej fixture w tests/conftest.py.


@pytest.fixture
def client(tmp_path, monkeypatch):
    monkeypatch.setenv("CACHE_PREFIX", "lab2:test:room-service")
    db_url = "sqlite:///" + (tmp_path / "test.db").as_posix()
    # MemoryRedis pochodzi z tests/test_storage.py z paczki pomocniczej.
    fake_redis = MemoryRedis()
    # Przykładowe przekazanie zależności — dostosuj do sygnatury create_app() ze swojego LAB1:
    app = create_app(database_url=db_url, redis_client=fake_redis)
    with TestClient(app) as test_client:
        yield test_client, fake_redis
```

Ręcznie sprawdź rzeczywisty Redis: MISS → HIT, wygaśnięcie TTL, restart Uvicorna oraz odłączenie i przywrócenie cache. Zapisz stan początkowy, żądanie, oczekiwany i rzeczywisty wynik oraz sposób resetu.

Używaj własnego prefiksu także w testach; nie wykonuj `FLUSHALL` ani `FLUSHDB` na wspólnym serwerze. Zamiennik z paczki nie symuluje upływu TTL, więc tego zachowania nie potwierdza.

Krok 8 — uzupełnienie dokumentacji projektu

Dopisz sekcje z `README-LAB2-template.md` , zachowując dokumentację LAB1:

- zmiany względem pierwszej wersji,
- model danych, właściciela zapisu i zachowaną regułę,
- konfigurację, ścieżkę bazy, uruchomienie i testy,
- odczyt objęty cache, klucz, TTL i unieważnianie,
- dowody trwałości, HIT/MISS, TTL i awarii,
- ograniczenia spójności i zastosowaną dokumentację,
- aktualizację planu drugiej usługi, wkładu autorów i użycia AI.

---

6. Porównanie pamięci procesu, bazy danych i cache

Cecha	Pamięć procesu z LAB1	SQLite	Redis jako cache
Rola	Stan procesu	Trwałe źródło prawdy	Odtwarzalna kopia
Restart Uvicorna	Zmiany znikają	Plik pozostaje	Osobny proces może zachować wpis
Brak danych	Stan nieutworzony lub utracony	Brak rekordu	MISS jest normalny
Trudność	Utrata zmian	Schemat, transakcje, błędy	Aktualność, TTL, awarie

Odpowiedz na przykładzie własnego projektu:

1. Co zmieniło się dla klienta API, a co wewnątrz usługi?
2. Dlaczego baza i cache nie zmieniają automatycznie aplikacji w system mikroserwisowy?
3. Czy druga usługa powinna pisać do tych samych tabel?
4. Jakie skutki ma awaria procesu aplikacji, Redis i bazy?
5. Jakie opóźnienie aktualizacji danych dopuszcza Twój odbiorca?
6. Co wymaga sprawdzenia przy zmianie SQLite na PostgreSQL?

Osobny właściciel danych nie wymaga osobnego fizycznego serwera. Istotne są kontrola zapisu i granica odpowiedzialności. Zmiana bazy wymaga sterownika, konfiguracji, schematu i testów; ORM nie gwarantuje pełnej zgodności.

---

7. Weryfikacja efektów i rozmowa techniczna

Punkt kontrolny A — zachowanie ciągłości projektu

Pokaż temat, API, regułę i plan drugiej usługi z LAB1. Uzasadnij zmiany.

Punkt kontrolny B — weryfikacja trwałości i reguły biznesowej

Pokaż zapis po restarcie i odrzuconą operację bez zmiany danych. Wskaż commit i rollback.

Punkt kontrolny C — weryfikacja działania cache

Pokaż MISS, HIT, TTL i aktualną odpowiedź po poprawnym unieważnieniu. Wyjaśnij skutki nieudanego unieważnienia.

Punkt kontrolny D — weryfikacja testów i obsługi awarii

Uruchom testy, wyjaśnij izolację i pokaż działanie API bez Redis.

Punkt kontrolny E — samodzielna modyfikacja kodu

Każda osoba wykonuje drobną zmianę, np. zmienia TTL i odpowiadającą mu asercję albo dodaje test brakującego zasobu. Wyjaśnia użycie dokumentacji i weryfikację kodu z AI.

---

8. Kryteria zaliczenia etapu LAB2

Punktacja dotyczy etapu projektu i, jak na LAB1, zachowuje proporcję kod 60 / dokumentacja 40. Nie zmienia warunków zaliczenia przedmiotu.

Obszar	Element	Maksimum
Kod	Trwała baza, zachowane API i reguła, błędy bez częściowego zapisu	20
Kod	Cache-aside, TTL, unieważnienie i obsługa awarii	20
Kod	Własne testy, asercje i izolacja	20
Dokumentacja	Ciągłość domeny, model i własność danych	10
Dokumentacja	Powtarzalna konfiguracja, uruchomienie i kontrakt API	10
Dokumentacja	Scenariusze i wyniki restartu, cache oraz awarii	10
Dokumentacja	Porównanie, ograniczenia, źródła, AI i autorzy	10
Razem	**Kod 60 + dokumentacja 40**	**100**

Zaliczenie etapu: minimum 50 punktów i wszystkie warunki minimum:

- działa usługa z LAB1 z health, listą, szczegółami, zapisem i regułą;
- przeniesione dane są spójnie odczytywane i zapisywane w bazie oraz przetrwają restart;
- odrzucone operacje nie zmieniają stanu;
- jeden odczyt ma cache z TTL i unieważnianiem;
- sprawny cache nie zwraca starej wartości po zakończonym sekwencyjnym scenariuszu zmiany (po usunięciu zasobu GET zwraca 404);
- awaria Redis pozwala na odczyt i zapis do bazy, a ograniczenia są opisane;
- przechodzą wymagane testy i wykonano ręczne sprawdzenie rzeczywistego Redis;
- README pozwala odtworzyć wynik, a każda osoba potrafi wyjaśnić i zmienić kod.

Brak warunku minimum oznacza konieczność poprawy mimo uzyskania 50 punktów.

---

9. Oznaczenie i przekazanie wersji projektu LAB2

Oddaj kolejną wersję **tego samego repozytorium** z `app` , `tests` , `requirements.txt` , README i `.gitignore` . Oznacz commit i tag etapu LAB2. Nie dołączaj środowiska, lokalnych baz, cache Pythona ani sekretów.

Przykładowa sekwencja poleceń Git:

```
git status # przed dodaniem sprawdź, czy nie ma baz, środowiska ani sekretów
git add .
git diff --cached --stat
git commit -m "feat(lab2): baza SQLite, cache-aside Redis, testy i obsluga awarii"
git tag lab2
git push origin HEAD
git push origin refs/tags/lab2
```

`HEAD` wysyła bieżącą gałąź (np. `main` albo `master`); pracuj na gałęzi, nie w stanie detached HEAD (`git branch --show-current` powinno wypisać nazwę). Wysyłamy tylko tag `lab2` , nie wszystkie lokalne tagi. Jeśli tag już istnieje, nie nadpisuj go siłą — uzgodnij nazwę poprawionej wersji, np. `lab2-v2` . Gdy push zostanie odrzucony, najpierw zintegruj zmiany zdalne; nie używaj `--force` .

Kod z paczki jest pomocą: oddawana wersja musi realizować własną domenę, regułę i testy. Nie wymagamy drugiej nowej aplikacji na tych zajęciach.

Dla chętnych: PostgreSQL, migracja Alembic, pomiar liczby zapytań i czasu, cache listy z filtrami lub test równoległego odczytu i zapisu. W LAB3 rozwiniesz dotychczasową usługę o rejestrację i odnajdywanie przez Eurekę.

---

10. Korzystanie z dokumentacji i asystentów AI (SQLAlchemy)

Status weryfikacji: 19.10.2026. W sprawdzonej oficjalnej dokumentacji SQLAlchemy i repozytorium projektu nie potwierdzono oficjalnego, gotowego do zainstalowania pliku `SKILL.md` do pracy z SQLAlchemy w aplikacji studenta. Nie znaleziono również głównego `AGENTS.md` w repozytorium. Nie traktuj skilli znalezionych w innych repozytoriach jako materiałów autorów SQLAlchemy.

Te nazwy oznaczają różne rzeczy:

Plik	Rola
`AGENTS.md`	Instrukcje pracy agenta w danym projekcie, np. zasady zmian i polecenia testów
`SKILL.md`	Instrukcja konkretnej umiejętności, ładowana przez narzędzie obsługujące skille
`llms.txt`	Jeśli serwis go udostępnia: materiał lub indeks dokumentacji dla modeli; nie jest instalowanym skillem

Obsługa i lokalizacja tych plików zależy od używanego narzędzia. Samo pobranie pliku albo nazwanie go `SKILL.md` nie gwarantuje, że agent go odczyta. Instrukcje z repozytorium biblioteki mogą dotyczyć rozwijania samej biblioteki, a nie używania jej w Twojej aplikacji.

Wykorzystanie oficjalnej dokumentacji podczas LAB2

1. Otwórz dokumentację SQLAlchemy 2.0, zgodną z zakresem `SQLAlchemy>=2.0,<2.1` przyjętym w materiałach.
2. W rozmowie z agentem przekaż odnośnik do konkretnego zagadnienia, np. sesji i transakcji. Poproś o przeczytanie strony przed zmianami. Jeśli narzędzie nie ma dostępu do strony, przekaż odpowiedni fragment jako kontekst.
3. Wskaż pliki własnej usługi i zachowania z LAB1, które mają zostać zachowane.
4. Po zmianie sprawdź diff, uruchom testy i opisz w README, jak zweryfikowałeś rozwiązanie.

Przykładowe polecenie do agenta:

> Przeczytaj https://docs.sqlalchemy.org/en/20/orm/session_basics.html. Rozwijamy usługę z LAB1 z SQLAlchemy 2.0. Najpierw porównaj dokumentację z naszym repozytorium i wskaż miejsce utworzenia, zatwierdzenia, wycofania i zamknięcia sesji. Zachowaj API, typy identyfikatorów i regułę biznesową. Zaproponuj małą zmianę oraz test wykazujący brak częściowego zapisu po błędzie. Wskaż fragment dokumentacji uzasadniający rozwiązanie. Jeśli nie możesz odczytać strony, powiedz o tym.

To przekazuje agentowi kontekst i konkretne kryteria pracy. Nie instaluje biblioteki, nie tworzy bazy i nie zastępuje testów.

Nie trzeba dodawać nowego pliku do projektu. Jeśli używany agent obsługuje projektowy `AGENTS.md` , możesz w głównym katalogu własnej usługi, obok README, zapisać własne zasady LAB2 i odnośniki do dokumentacji. Jeśli plik już istnieje, uzupełnij go zamiast nadpisywać. Oznacz te zasady jako instrukcje projektu studenckiego, nie oficjalny materiał SQLAlchemy. Sposób automatycznego odczytu sprawdź w dokumentacji swojego narzędzia.

Ocena nieoficjalnych materiałów do pracy z bazą danych (SQLAlchemy)

Poniższe materiały są tworzone przez społeczność, **nie przez zespół SQLAlchemy**. To propozycje do samodzielnej oceny, a nie wymagane zależności LAB2. Sprawdzono ich treść 19.10.2026; linki do gałęzi `main` mogą później prowadzić do zmienionych wersji.

Skill i autor	Co obejmuje	Jak wykorzystać na LAB2
sqlalchemy — bm629/agent-skills	Modele SQLAlchemy 2.x, sesje, transakcje, podejście synchroniczne i różnice między bazami	Pomoc przy przeglądzie repozytorium i przenoszeniu danych do SQLite. Zawiera odwołania do plików w `references/` ; zachowaj je przy pobieraniu całego skilla
sqlalchemy-expert — SKB3002/claude-code-kit	Głównie wariant asynchroniczny, sesje, zapytania, relacje i migracje Alembic	Materiał porównawczy lub rozszerzający. Na LAB2 zachowujemy wariant synchroniczny; nie przechodź na async tylko dlatego, że preferuje go skill

Uwaga: skill zawiera instrukcje dla agenta i może wpływać na edycję plików oraz wykonywanie poleceń. Może też zawierać błędy, uproszczenia albo zalecenia niezgodne z wersją bibliotek w Twoim projekcie. Nie jest gwarancją poprawnego kodu ani audytowanym dodatkiem. Przed użyciem przeczytaj `SKILL.md` , powiązane instrukcje i ewentualne skrypty. Nie uruchamiaj bez sprawdzenia instalatorów, nie przekazuj sekretów i nie akceptuj kasowania bazy jako sposobu naprawy migracji. W razie sprzeczności sprawdź oficjalną dokumentację i zachowaj wymagania LAB2.

Pobranie i konfiguracja wybranej instrukcji agenta AI

Wybierz jeden materiał; nie musisz instalować obu.

1. Otwórz repozytorium z tabeli. Na GitHub użyj historii lub klawisza `y` , aby uzyskać odnośnik do konkretnego commita. Zapisz go w README — ułatwia to odtworzenie użytej wersji.
2. Pobierz katalog wybranego skilla, zachowując `SKILL.md` i potrzebne pliki pomocnicze oraz informacje o licencji. W pierwszej propozycji uwzględnij `references/` . Nie trzeba instalować całej kolekcji.
3. Umieść katalog w miejscu obsługiwanym przez Twojego agenta. Na przykład **Claude Code** obsługuje projektowe skille w `.claude/skills/<nazwa>/SKILL.md` , zgodnie z dokumentacją skilli Claude Code:

```
my-service/
├── .claude/
│   └── skills/
│       └── sqlalchemy/
│           ├── SKILL.md
│           └── references/
├── app/
├── tests/
└── README.md
```

Dla drugiej propozycji użyj katalogu `sqlalchemy-expert` . To przykład dla Claude Code, nie uniwersalna ścieżka dla wszystkich agentów.

1. W Claude Code wywołaj `/sqlalchemy` albo `/sqlalchemy-expert` . Jeśli skill nie jest wykrywany, sprawdź położenie i pole `name` w nagłówku pliku. Inne narzędzia mogą mieć inny sposób rejestracji i wywołania.
2. Jeśli narzędzie nie obsługuje skilli, zachowaj pliki np. w `docs/ai/sqlalchemy/` i jawnie poproś o przeczytanie `docs/ai/sqlalchemy/SKILL.md` . To przekazanie kontekstu, a nie automatyczna instalacja skilla.

Zacznij od przeglądu kodu, na przykład:

> Użyj skilla sqlalchemy do przeglądu warstwy danych mojego projektu z LAB1. Pracujemy synchronicznie, z SQLite i SQLAlchemy 2.0. Zachowaj API, identyfikatory i regułę biznesową. Najpierw wskaż problemy z sesjami i transakcjami oraz zaproponuj testy. Porównaj zalecenia skilla z oficjalną dokumentacją. Nie zmieniaj jeszcze plików ani schematu bazy.

Skill ma pomóc agentowi stosować określony sposób pracy. Sam nie instaluje SQLAlchemy, nie uruchamia Redis i nie weryfikuje automatycznie poprawności projektu. Dopiero kolejne polecenie może zlecić implementację; sprawdź wtedy diff i uruchom własne testy.

W README zapisz autora, odnośnik do wersji, zakres wykorzystania, zauważone rozbieżności i sposób weryfikacji. Użycie skilla nie zastępuje dwóch wymaganych fragmentów oficjalnej dokumentacji.